TCSS 321 – Discrete Structures

Loop Analysis

Counting Operations

```
Minimum (A = (a_1, a_2, ..., a_n))
    Let m = a_1.
    For i = 2 to n:
        If a_i < m then let m = a_i.
    Endfor
Return m.
```

- How many operations does this algorithm take? To answer this we must answer a few questions.
  - What counts as an operation?
  - How big is n?
  - How do we handle the two cases when $a_i < m$ or not?

Operations

`Minimum` $(A = (a_1, a_2, \ldots, a_n))$	1 method call
`Let` $m = a_1$.	1 assignment operation
`For` $i = 2$ `to` n:	1 assignment or increment + 1 comparison
`If` $a_i < m$ `then let` $m = a_i$.	1 comparison + (possibly) 1 assignment
`Endfor`	0 operations
`Return` m.	1 operation

- What counts as an operation determines the resolution of our analysis.
  - A fine or high resolution analysis might count every single bit operation.
  - A mid level resolution might count the number of constant time Java operations.
  - A rough or low level resolution might only count one type of operation.
- Common operations that are counted are declarations, assignments, comparisons, output, logical and arithmetic operators, or method calls.

Input Size

```
Minimum (A = (a_1, a_2, ..., a_n))
        Let m = a_1.                                  1 method call
        For i = 2 to n:                               1 assignment operation
                If a_i < m then let m = a_i.          1 assignment or increment + 1 comparison
        Endfor                                        1 comparison + (possibly) 1 assignment
Return m.                                             0 operations
                                                      1 operation
```

- Assume $n = 1$. How many operations are carried out?
  - 1 call + 2 assignments + 1 comparison + 1 return = 5 operations
- Assume $n = 2$. How many operations are carried out?
  - Best case: 1 call + 2 assignments + 2 comparisons + 1 increment + 1 comparison + 1 return = 8 operations
  - Worst case: 1 call + 2 assignments + 2 comparisons + 1 assignment + 1 increment + 1 comparison + 1 return = 9 operations
  - In worst case analysis we always assume that during a branch the more costly option is executed.
- We can see that the number of operations depends on the input size n.

Functions Revisted

Definition: A function, f, maps from a set D called the domain to a set R called the co-domain, denoted $f: D \rightarrow R$, such that $f(d) = r$ for $d \in D$ and $r \in R$.

- The domain and co-domain are like the variable types of the argument and return variable of a Java function.

```
int square(int n) {
        return n*n;
}
```

- Here square maps from the integers to the integers. Let f express this function then:

$$f: \mathbb{Z} \rightarrow \mathbb{Z}$$

- We can be more precise by noting that every integer can be used as an argument but not every integer is returned. Indeed:

$$f: \mathbb{Z} \rightarrow \mathbb{N}$$

Runtime Functions

- In order to express the number of operations carried out by an algorithm on an input of unknown size we need a runtime function.

Definition: A runtime function expresses the number of operations carried out by an algorithm as a function $f: \mathbb{N} \to \mathbb{N}$ of the input size n.

- Since a negative number of operations does not make sense runtime functions are typically positive (technically non-negative).

Definition: A function $f(n)$ is positive if and only if $f(n) > 0$.

Definition: A function $f(n)$ is non-negative if and only if $f(n) \geq 0$.

- Since the larger input usually makes the problem harder runtime functions are typically increasing (technically non-decreasing).

Definition: A function $f(n)$ is increasing if and only if $f(n) < f(n+1)$.

Definition: A function $f(n)$ is non-decreasing if and only if $f(n) \leq f(n+1)$.

Loop Analysis

```
Minimum (A = (a_1, a_2, ..., a_n))
    Let m = a_1.
    For i = 2 to n:
        If a_i < m then let m = a_i.
    Endfor
Return m.
```

- How many assignment operations does this algorithm use in the worst case?
  - There are 2 assignments prior to the loop and then we need to count the number of assignments in the loop.
  - It is straightforward to see that the loop will make $n-1$ assignments in the worst case.
  - So in total we can say that this algorithm makes $f(n) = 2 + n - 1 = n + 1$ assignment operations in the worst case.

Loop Analysis with Sums

```
Minimum (A = (a_1, a_2, …, a_n))
        Let m = a_1.
        For i = 2 to n:
                If a_i < m then let m = a_i.
        Endfor
Return m.
```

- It is common to represent the number of operations carried out by a loop as a sum of the number of operations in each iteration of the loop.
  - In our loop we only make one assignment in the worst case for each iteration. So the cost is:

$$\overbrace{1 + 1 + \cdots + 1}^{n-1}$$

- Using summation notation we can write this sum as:

$$\sum_{i=2}^{n} 1 = \sum_{i=1}^{n-1} 1 = \overbrace{1 + 1 + \cdots + 1}^{n-1}$$

Definition: A summation (or series), $\sum_{i=1}^{n} a_i = a_1 + a_2 + \cdots + a_n$.

Simple Summations

Theorem:

$$\sum_{i=1}^{n} 1 = n$$

Proof: Let $n \in \mathbb{N}$. Then

$$\sum_{i=1}^{n} 1 = \overbrace{1 + 1 + \cdots + 1}^{n} = n \cdot 1 = n \blacksquare$$

Theorem:

$$\sum_{i=a}^{b} 1 = b - a + 1$$

Proof: Let $n \in \mathbb{N}$. Then

$$\sum_{i=a}^{b} 1 = \sum_{i=a-a}^{b-a} 1 = \sum_{i=0}^{b-a} 1 = \sum_{i=1}^{b-a+1} 1 \blacksquare$$

Theorem:

$$\sum_{i=1}^{n} d \cdot a_i = d \cdot \sum_{i=1}^{n} a_i$$

- This theorem follows immediately from the distributive laws of arithmetic.

Finer analysis

```
Minimum (A = (a_1, a_2, ..., a_n))
        Let m = a_1.                                  1 method call
        For i = 2 to n:                               1 assignment operation
                                                      1 assignment or increment + 1 comparison
                If a_i < m then let m = a_i.          1 comparison + (possibly) 1 assignment
        Endfor                                        0 operations
Return m.                                             1 operation
```

- We already counted the number of assignment operations. Can we do a finer analysis and count all the relevant operations?
- Prior to the loop we have 2 operations.
- The loop can be expressed as a summation:

$$\sum_{i=2}^{n} 4$$

- After the loop we have 2 operations to complete the loop plus 1 additional operation.

Finer analysis

```
Minimum (A = (a_1, a_2, ..., a_n))
        Let m = a_1.                                  1 method call
        For i = 2 to n:                               1 assignment operation
                If a_i < m then let m = a_i.          1 assignment or increment + 1 comparison
        Endfor                                        1 comparison + (possibly) 1 assignment
Return m.                                             0 operations
                                                      1 operation
```

- So the total operations are:

$$2 + \sum_{i=2}^{n} 4 + 3 = 5 + \sum_{i=2}^{n} 4$$

$$= 5 + 4\sum_{i=2}^{n} 1$$

$$= 5 + 4(n-1)$$

$$= 5 + 4n - 4$$

$$= 4n + 1$$

Loop Analysis

```
Search (A = (a_1, a_2, ..., a_n), t)
    For i = 1 to n:
        If a_i = t then return i.
    Endfor
Return −1.
```

1 method call
1 assignment or increment + 1 comparison
1 comparison
0 operations
1 operation

- How many operations are there on each line of this code?
- Can we express the cost of the whole method using a summation?

$$f(n) = 1 + \sum_{i=1}^{n} 3 + 3 = 4 + \sum_{i=1}^{n} 3 = 4 + 3\sum_{i=1}^{n} 1 = 3n + 4$$

Loop Analysis

```
Equals (A = a_1 a_2 ⋯ a_n, B = b_1 b_2 ⋯ b_m)
        If n ≠ m return false.                          1 method call
        For i = 1 to n:                                 1 comparison
                If a_i ≠ b_i then return false.         1 assignment or increment + 1 comparison
        Endfor                                          1 comparison
Return true.                                            0 operations
                                                        1 operation
```

- How many operations are there on each line of this code?
- Can we express the cost of the whole method using a summation?

$$f(n) = 2 + \sum_{i=1}^{n} 3 + 3 = 5 + \sum_{i=1}^{n} 3 = 5 + 3\sum_{i=1}^{n} 1 = 3n + 5$$

Arithmetic Series

- The term of a sum can vary with the index just as the cost of one iteration of a loop may change as the loop progresses.
- A common and well-known sum is an arithmetic series:

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n$$

Theorem:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

- We can prove this theorem in using a couple different strategies.
  - We will practice mathematical induction first.
  - Then we will use a direct proof due to mathematician Gauss.
  - Finally we will prove this using geometry as the Greeks did.

Arithmetic Series

Theorem:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Proof: (by induction on n)

Base case: $(n = 1)$

$$\sum_{i=1}^{1} i = 1$$

$$\frac{1(1+1)}{2} = \frac{2}{2} = 1$$

Inductive Hypothesis: Assume $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$.

Arithmetic Series

Proof: (continued)

Inductive Hypothesis: Assume $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$ for some $n \geq 0$.

Inductive Step: $(n \geq 1)$

We need to show that $\sum_{i=1}^{n+1} i = \frac{n+1(n+2)}{2}$.

$$\sum_{i=1}^{n+1} i = (n+1) + \sum_{i=1}^{n} i$$

$$= (n+1) + \frac{n(n+1)}{2}$$

$$= \frac{2(n+1) + n(n+1)}{2}$$

$$= \frac{(n+2)(n+1)}{2}$$

By induction on n we have shown that $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$ ■

Arithmetic Series

Theorem:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Proof: (direct proof)

Notice that if you add the first and last elements of this series you get $n+1$. This is true for the second and second last elements too. We can pair up all elements in this way and always get the sum $n+1$. We have n terms so there are $\frac{n}{2}$ pairs. Thus,

$$\sum_{i=1}^{n} i = \frac{n}{2}(n+1) = \frac{n(n+1)}{2}$$

■

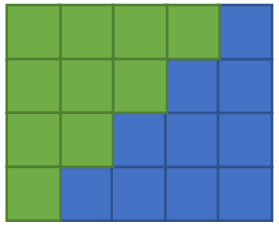
Arithmetic Series

Theorem:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Proof: (geometric proof)

The area of this shape is equal to $\sum_{i=1}^{4} i$.



Notice that if we make a copy of this shape and rotate it we can fit them together to get a rectangle.

This area of the rectangle is $n(n+1)$. Since have used two copies of the shape the shape must have area $\frac{n(n+1)}{2}$. ■

Elegance

- We have offered three different proofs of the same theorem.
  - Induction, direct proof, and geometric proof.

- Which proof is best?
  - We can apply the same tests to proofs as we do algorithms.
  - First, is the proof correct? That is, is it sound and valid?
  - Second, how "efficient" is the proof? That is, how short is it and how simple is it to understand?

- When a proof is short and easy to understand we often say it is an elegant proof.
  - We prefer elegant proofs!

Geometric Series

- Another common well-known sum is the geometric series:

$$\sum_{i=0}^{n} 2^i = 2^0 + 2^1 + \cdots + 2^n$$

- This sum is known to have a closed form expression

$$\sum_{i=0}^{n} 2^i = 2^{n+1} - 1$$

- Can we prove this by induction?

Geometric Series

Theorem:

$$\sum_{i=0}^{n} 2^i = 2^{n+1} - 1$$

Proof: (by induction on n)

Base case: ($n = 0$)

$$2^0 = 1 = 2^1 - 1$$

Inductive Hypothesis: Assume $\sum_{i=0}^{n} 2^i = 2^{n+1} - 1$ for some $n \geq 0$.

Inductive Step: ($n \geq 0$)

$$\begin{aligned}\sum_{i=0}^{n+1} 2^i &= 2^{n+1} + \sum_{i=0}^{n} 2^i \\ &= 2^{n+1} + 2^{n+1} - 1 \\ &= 2 \cdot 2^{n+1} - 1 \\ &= 2^{n+2} - 1\end{aligned}$$

By induction we have shown that

$$\sum_{i=0}^{n} 2^i = 2^{n+1} - 1$$

■